GPU Sustained-Capacity Impairment Watch Memo

Memo: GPU Sustained-Capacity Impairment Watch

Branch: `codex/gpu-sustained-capacity-impairment-watch`

Snapshot date: 2026-06-03

Short Version

This branch turns the loose "compute headroom" idea into a narrower ARC-ready receipt primitive.

It does not claim to predict how much future work a GPU node can safely absorb. It asks a more bounded question:

```
During this declared window, on these bound GPUs, under meaningful observed
load, did the telemetry remain consistent with no sustained-capacity impairment
across declared clock, throttle, thermal, power, ECC, Xid, and fabric
thresholds?
```

That narrower claim is useful because it is objective enough for ARC's current scanner -> evidence gap -> deterministic pass -> bounded receipt loop. It also keeps the demo aligned with the finance-grade receipt thesis: a lender, marketplace, or insurer does not need another dashboard; it needs portable, bounded evidence about the condition of the compute asset or service during a financially relevant window.

Why "Headroom" Was Too Loose At First

"Compute headroom" is intuitively the remaining useful load a node or rack can take before the limiting factor becomes power delivery, thermal throttling, memory errors, GPU clocks, or fabric degradation.

That is a valuable product idea, but as stated it is not yet a clean receipt claim. A receipt needs a bounded proposition and a deterministic way to decide `supported`, `contradicted`, or `unknown` from supplied evidence. A predictive headroom score would require historical baselines, workload class, forecast assumptions, and calibration against real failure or throttling outcomes. The current ARC demo prep project is not ready to support that claim honestly.

So this branch separates the product north star from the first receiptable primitive:

```
Product direction:
  sustained compute headroom

Current branch:
  observed sustained-capacity impairment watch
```

The distinction matters. A clean window under meaningful load can support "no observed impairment signals crossed these thresholds during this window." It cannot by itself support "this node has 37% spare capacity tomorrow."

Headroom As A Bridge Metric

The better product framing is not "Ashiba invented headroom." It did not. Headroom language is already common across server telemetry, power management, DCIM, and finance.

The more precise framing is:

`Ashiba is exploring whether GPU/rack headroom can become a receipt-grade bridge metric for outside financial counterparties.`

In ordinary infrastructure tooling, headroom answers an operator question:

`How much cushion remains before this system hits a power, thermal, bandwidth, or performance limit?`

In finance, headroom answers a covenant question:

`How much cushion remains before this borrower or asset breaches a threshold?`

The branch tries to connect those two ideas. It takes low-level GPU/rack signals and begins shaping them into a bounded artifact that a lender, marketplace, or insurer could use without becoming the operator of the cluster.

That means the actual bridge is:

`operator telemetry -> limiting-dimension margins -> bounded receipt -> finance / settlement / covenant action`

The current branch stops at the deterministic receipt primitive. It does not yet compute a single scalar "headroom score." But it defines the limiting dimensions that such a score would need to respect, and it preserves the most important ARC behavior: if the evidence is weak, low-load, missing, malformed, or unbound, the output is `unknown`, not a comforting number.

Nearest Headroom Neighbors

The nearest public neighbors make the branch's boundary sharper.

Dell iDRAC compute headroom

Dell's iDRAC telemetry documentation includes a server-level `SystemUsagePctReading` that aggregates CPU, memory, and I/O utilization and describes it as a measurement of "the compute headroom available on the server." This is a direct headroom neighbor.

The gap: Dell's metric is server-local operational telemetry. It is not a GPU/rack financial receipt, and it does not appear to bind a financially relevant time window, node schedule, GPU UUID set, threshold declaration, and counterparty-facing verdict.

Source: https://www.dell.com/support/manuals/en-us/poweredge-t360/idrac_telemetry_reference_guide_pub/cumulative-system-usage

Modius rack headroom

Modius describes AI DCIM for GPU-heavy data centers as normalizing telemetry from sources such as Redfish and time-series monitoring systems, then computing derived metrics including rack headroom and GPU throttling.

The gap: this is operator-side DCIM and capacity planning. It is very close to the infrastructure metric, but not the same as a portable ARC receipt for an outside lender, insurer, or marketplace.

Source: https://modius.com/blog/dcim-for-ai-designing-power-cooling-and-observability-for-gpu-heavy-data-centers/

Power and thermal headroom prior art

Power headroom, thermal headroom, TDP headroom, rack power constraints, and task scheduling based on available headroom are crowded. There are old patents and ordinary engineering practices around using remaining thermal or power budget to control frequency, voltage, throttling, scheduling, and platform power allocation.

The gap: these systems generally use headroom for control. Ashiba is not claiming the control loop. The branch uses headroom-adjacent telemetry to compile an evidentiary artifact about a bounded window.

Example source: https://patents.google.com/patent/US11703930B2/en

CapitalBridge covenant headroom

CapitalBridge is not a compute-infrastructure neighbor, but it is an excellent finance-language neighbor. It frames covenant headroom as the remaining cushion before a covenant breach, with status changes before the binary breach event.

The gap: CapitalBridge works on financial covenants such as DSCR, LTV, ICR, and asset cover. It does not translate GPU/rack telemetry into a physical-capacity margin.

Source: https://capitalbridge.app/covenant-headroom-monitoring

Symmetric Research SCU

Symmetric Research is the closest compute-finance neighbor. Its public description says it builds independent collateral verification for GPU-backed lending and has a Standard Compute Unit that normalizes heterogeneous GPU fleets into a single comparable metric.

The gap: SCU appears to be a comparability/unitization metric for compute capital. The Ashiba branch is narrower and more evidentiary: did a bound node show observed sustained-capacity impairment during a declared window under meaningful load?

Source: https://www.linkedin.com/company/symres

Aravolta telemetry underwriting

Aravolta is the closest commercial GPU-finance telemetry neighbor. Its public materials describe real-time GPU utilization, temperature, power draw, memory usage, workload patterns, thermal violations, power spikes, lifecycle tracking, asset risk, and lender dashboards.

The gap: Aravolta appears to be visibility, underwriting, and dashboard oriented. Ashiba's possible wedge is not that it sees unique telemetry. It is that it can turn a narrow claim about that telemetry into a fail-closed receipt artifact.

Source: https://www.aravolta.com/lenders/telemetry-underwriting

What Is Actually Differentiated

The branch should not imply that headroom metrics are new. They are not.

The differentiated hypothesis is narrower:

```
Make headroom receipt-grade for finance and settlement.
```

That means the branch is testing whether ARC can convert headroom-adjacent operator evidence into something with these properties:

- the window is declared;
- the GPUs are bound to a node and GPU UUID set;
- the limiting dimensions are named;
- thresholds are explicit;
- low-load evidence cannot support the claim;
- missing or malformed evidence returns `unknown`;
- threshold violations return `contradicted`;
- a clean window supports only the bounded claim, not future capacity.

This is why the current implementation uses an impairment watch instead of a positive headroom score. The safest first bridge metric is not:

```
remaining safe capacity = 37%
```

It is:

```
the observed window did not cross declared impairment thresholds under
meaningful load
```

That is less exciting, but it is more receipt-shaped. A later branch can derive a scalar headroom score from the same dimensions if the evidence base and buyer need justify it.

How We Got Here

The ARC project already has a strong product discipline:

- Receipts evaluate narrow operational claims against supplied evidence.
- Missing, malformed, or insufficient evidence returns `unknown`.
- Evidence conflicts return `contradicted`.
- `supported` must not imply custody, authenticity, future reliability, general system health, or semantic correctness.
- The boundary language is part of the product, not boilerplate.

The GPU finance thread pushed that discipline into a concrete buyer problem. For a GPU lender, compute marketplace, or insurer, ordinary utilization is a weak signal. A GPU can be utilized while it is thermally throttling, clocked below expectation, near a power limit, emitting ECC/Xid errors, or sitting behind degraded fabric. Conversely, idle clean telemetry proves very little about sustained capacity.

The useful product move was to stop asking whether Ashiba could make a broad "healthy capacity" claim and instead ask:

```
What telemetry dimensions would make a financially relevant capacity claim
decidable, contradicted, or unknown?
```

This branch encodes the first answer in the receipt compiler.

What The Branch Adds

The branch adds the claim pack:

```
claim_packs/gpu_sustained_capacity_impairment_watch.json
```

The claim text is:

```
The bound GPU node showed no observed sustained-capacity impairment signals
across declared clock, throttle, thermal, power, ECC, Xid, and fabric thresholds
during the measurement window under meaningful load.
```

The deterministic pass consumes three evidence groups:

- `declaration`: the measurement window and declared thresholds.
- `gpu_impairment_binding`: the node and GPU UUIDs under review.
- `gpu_impairment_window`: timestamped telemetry samples for those GPUs.

The dimensions checked are:

- sample count inside the declared window;
- mean GPU utilization, used as a meaningful-load floor;
- minimum SM clock ratio;
- throttle sample fraction;
- minimum thermal margin;
- minimum power margin;
- uncorrectable ECC delta;
- Xid count delta;
- fabric error delta.

The pass returns:

- `supported` when all required evidence is present, samples are bound to the declared GPUs, observed load is high enough, and all dimensions stay within declared thresholds;
- `contradicted` when bound evidence crosses a declared impairment threshold or sample UUIDs do not match the bound GPU set;

- `unknown` when evidence is missing, malformed, outside the window, too sparse, or collected under too little load.

The examples added on the branch cover all three behaviors:

```
examples/gpu_impairment_supported
examples/gpu_impairment_contradicted_thermal
examples/gpu_impairment_contradicted_fabric
examples/gpu_impairment_unknown_idle
examples/gpu_impairment_unknown_missing_clock
```

Why This May Make Sense Inside ARC Demo Prep

ARC's demo shape is strongest when it can show that a high-stakes claim is not ready until the evidence is bound tightly enough. This branch gives the GPU finance story that same shape.

The demo progression can be:

```
1. A lender or marketplace wants to know whether a pledged/reserved GPU node
   exhibited sustained-capacity impairment during a billing or covenant window.

2. ARC scans the evidence packet.

3. If the packet only has generic utilization, generic dashboard status, or
   unbound telemetry, ARC returns a gap rather than a decorative receipt.

4. Once the packet includes GPU binding, declared thresholds, timestamped
   samples, and enough meaningful load, ARC compiles a bounded receipt.

5. The receipt can support, contradict, or leave unknown the narrow impairment
   claim without pretending to prove future capacity, collateral authenticity,
   or operator honesty.
```

That is the same core product pattern as the authorization demo: the user does not have to adopt a new logging standard, but ARC tells them what evidence is missing before a claim can be decided.

Why This Is Better Than Utilization

Utilization is a load indicator, not a capacity-quality indicator.

A high-utilization node can still be bad collateral or a bad rented service if it is throttled, power constrained, clock degraded, erroring, or fabric impaired. A low-utilization node can look clean while proving almost nothing about whether it could sustain load. In this branch, utilization is only the gate that says the window was meaningful enough to interpret the other signals.

The receipt therefore asks a more useful question:

```
Given that the GPUs were actually under meaningful load, did any limiting
capacity dimension exhibit an impairment signal?
```

This is still not a full goodput benchmark and not a forecast. But it is a better evidence primitive than raw utilization because it connects load to the subsystems that determine whether capacity is actually usable.

Buyer Use Cases

The immediate buyer-facing use cases are not "optimize my cluster." They are financial and settlement uses where a portable bounded artifact is valuable.

A GPU lender could use the receipt as:

- a covenant evidence artifact for collateral monitoring;
- a cure/default triage input when capacity impairment appears during a pledged operation window;

- a periodic audit artifact that is more concrete than borrower dashboard screenshots.

A compute marketplace could use it as:

- settlement evidence for a disputed reservation or billing window;
- a provider-quality artifact alongside availability and job-completion data;
- a way to distinguish "the node was busy" from "the node sustained usable capacity without observed impairment."

A compute insurer could use it as:

- underwriting evidence for operational risk;
- claims triage evidence after a reported service degradation;
- a structured way to decide whether missing telemetry leaves the claim `unknown` rather than over-resolved.

What This Branch Must Not Claim

This branch should not be marketed or documented as proving:

- future headroom;
- residual collateral value;
- node authenticity or custody;
- firmware authenticity;
- workload correctness;
- cluster-level goodput;
- SLA compliance across unobserved windows;
- absence of all faults;
- operator honesty;
- insurance-grade causation.

The honest branch sentence is:

```
Adds an objective GPU sustained-capacity impairment receipt, using bound
telemetry windows to detect observed thermal, power, clock, error, and fabric
degradation under meaningful load.
```

How This Fits With The Future Predictive Version

The future predictive version should be a separate layer built on top of this one, not a hidden interpretation of this branch.

A predictive sustained-capacity headroom claim would need at least:

- repeated historical windows per node and rack;
- workload-class labeling;
- baseline distributions for clocks, power, thermal margin, ECC/Xid, and fabric counters;
- calibration against observed throttling, job slowdown, drain events, or operator incidents;
- declared forecast horizon;
- confidence intervals and conditions under which the model returns `unknown`.

The current branch is still useful because it creates the deterministic feature surface that the future model would depend on. It also lets ARC practice the most important product behavior now: refusing to support a claim when the evidence is too weak.

PR Shape Recommendation

This branch is currently stacked on the GPU power-utilization consistency work. If opened directly against `main`, GitHub will show both that earlier power branch and this impairment-watch change. The cleaner review shape is:

```
PR 1:
  gpu_power_utilization_consistency
```

```
PR 2:
  gpu_sustained_capacity_impairment_watch
  based on PR 1, or rebased onto main after PR 1 lands
```

The PR title should avoid "compute headroom" and use the exact implemented claim:

```
Add GPU sustained-capacity impairment watch receipt
```

The PR body can mention compute headroom as the product direction, but the code claim should stay narrower.

Bottom Line

The branch makes sense inside ARC demo prep because it converts a promising but overbroad product idea into a receipt-shaped claim.

It gives the demo a concrete GPU finance artifact:

```
not dashboard telemetry
not generic utilization
not predictive capacity magic
but a bounded receipt over bound telemetry, declared thresholds, and a
financially relevant measurement window
```

That is exactly the kind of surface ARC is supposed to make legible.